

Sure! Let's clearly break down how **dynamic programming** is used in your solution, and where the core DP technique shows up.

What's the problem?

You're finding the **minimum-cost path** from node A to node J on a **directed acyclic graph (DAG)** represented by a weighted adjacency matrix. All edges go forward (i.e., from lower index to higher index).

Where Dynamic Programming Comes In

◆ 1. Subproblem Definition

Let `dp[i]` be the **minimum cost to reach the last node (J)** starting from node `i`.

◆ 2. Base Case

Js

Copy

```
dp[n - 1] = 0; // cost to reach the end from the end is zero
```

◆ 3. Recurrence Relation

For every node `i`, check all outgoing edges `i → j`, and update `dp[i]` based on the cost to reach `j`.

Js

Copy

```
dp[i] = min( data[i][j] + dp[j] ) // for all j > i where data[i][j] > 0
```

This computes the best choice of next step `j`, adding its total remaining cost.

◆ 4. Order of Computation

You go **from right to left** (high to low index). This ensures all future costs `dp[j]` are already computed when evaluating `dp[i]`.



Path Reconstruction

Once all costs `dp[i]` are computed, you **reconstruct the path** by choosing the next node that gave you the lowest cost. That's stored in:

Js

Copy

```
nextHop[i] = j; // the best j to go to from i
```



Dynamic Programming Characteristics

Feature	How it's used here
Optimal Substructure	Best path from <code>i</code> relies on best path from <code>j</code>
Overlapping Subproblems	Multiple <code>i</code> values may rely on same <code>dp[j]</code>
Memoization/Tabulation	<code>dp[i]</code> stores the result and avoids recomputation
Topological Order	Right-to-left ensures dependencies are resolved



Example of Recurrence (in code)

Js

Copy

```
for (let i = n - 2; i >= 0; i--) {  
  dp[i] = Infinity;  
  for (let j = i + 1; j < n; j++) {  
    if (data[i][j] > 0) {  
      const newCost = data[i][j] + dp[j];  
      if (newCost < dp[i]) {  
        dp[i] = newCost;  
        nextHop[i] = j;  
      }  
    }  
  }  
}
```

You build solutions from smaller ones — classic DP.

